Recursive Algorithms

Part 3: Maze Solving Algorithm

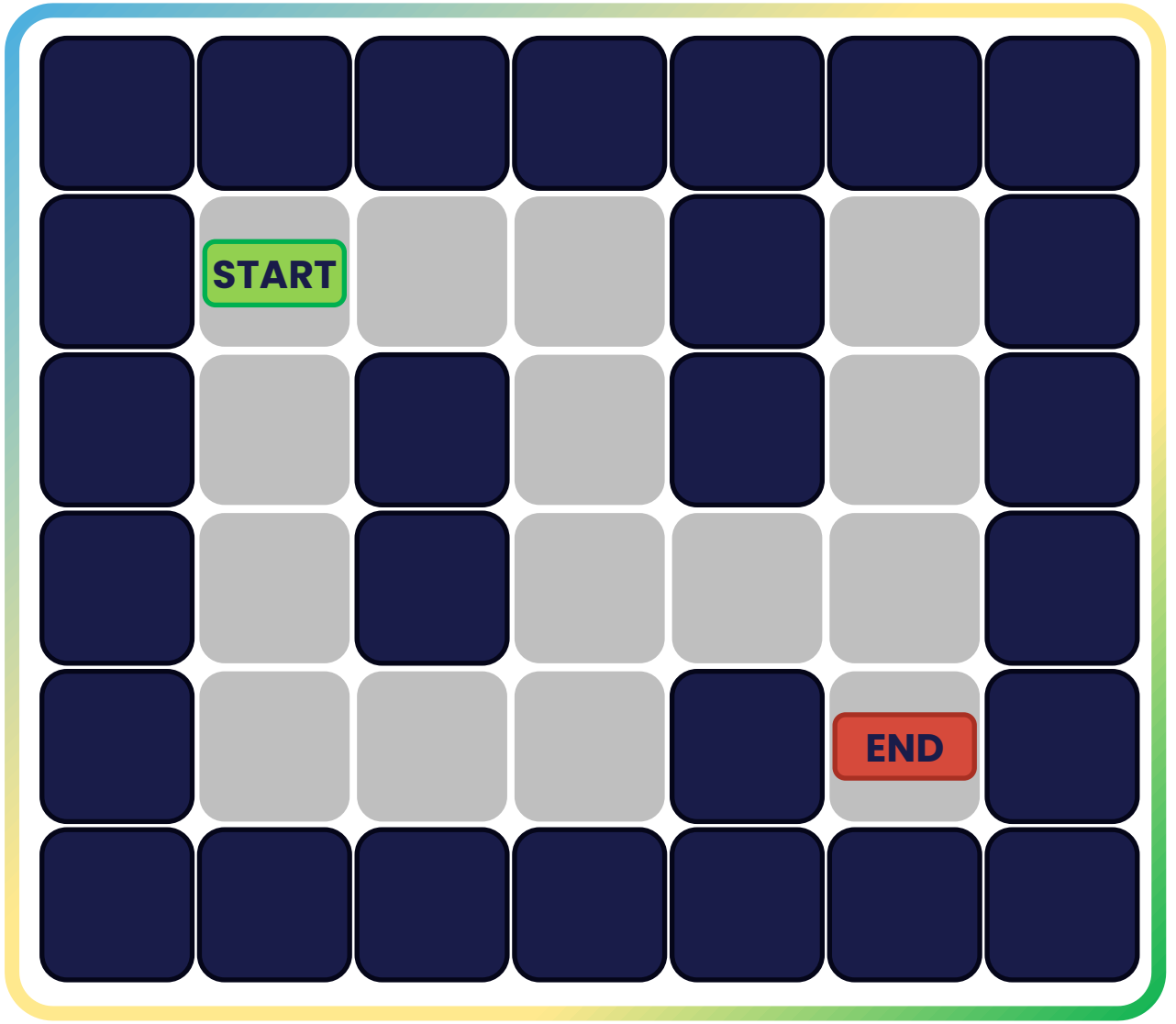
Consider a Maze

This is a pretty simple maze that you could probably solve quite easily!

But how would you tell a computer to solve it?

What rules could we create that would solve every maze, every time?

*(There are technically many answers to this question – but a recursive solution is called **Depth First Search**)*

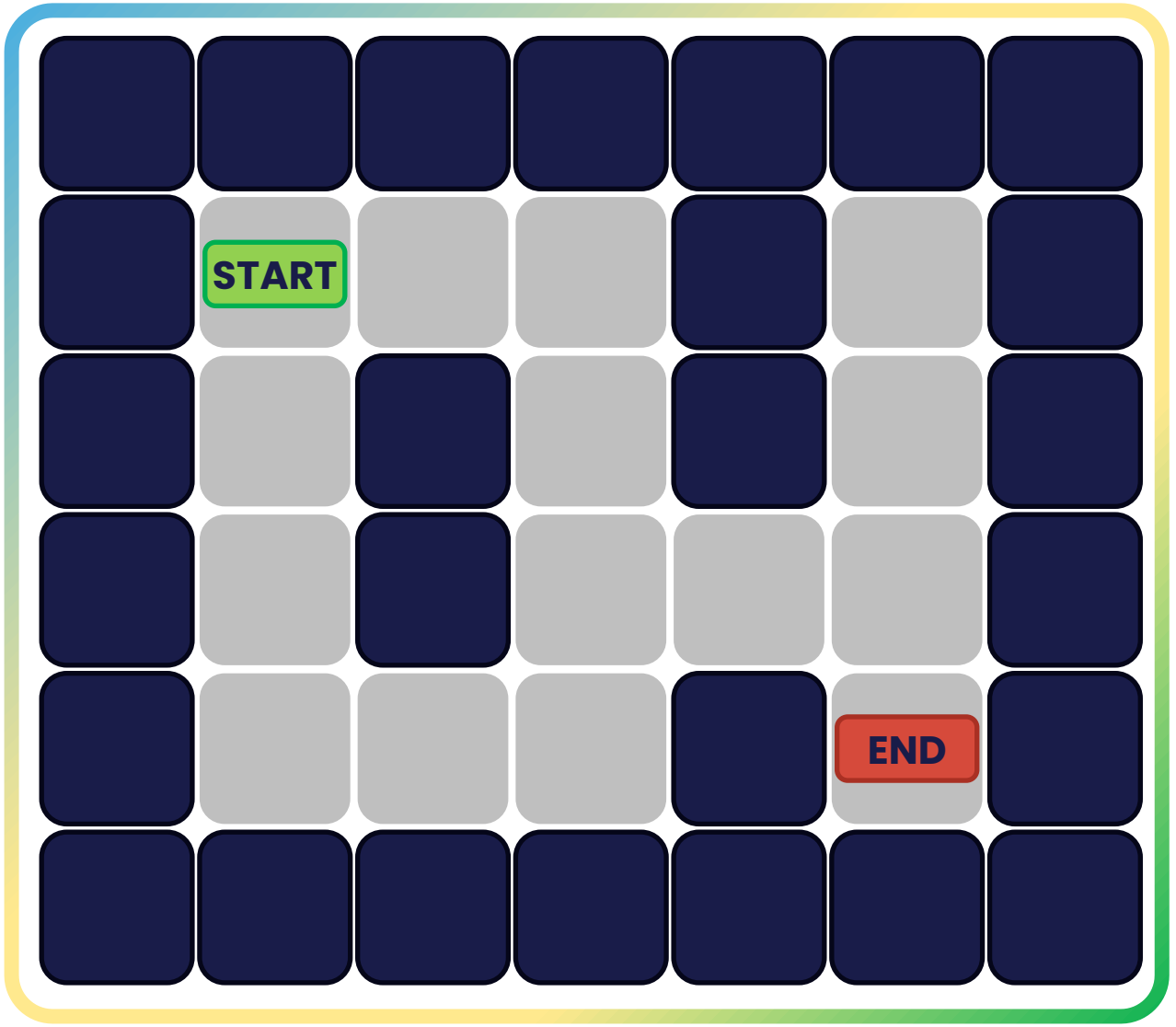


How Could We Solve It?

The algorithm follows a simple process:

1. Check whether the current position is the exit.
2. Check whether the current position is invalid (a wall, outside the maze, or already visited).
3. Mark the current position as visited.
4. Recursively attempt to solve the maze from each neighbouring position.
5. If one direction leads to the exit, return True.
6. If every direction fails, return False.

The important idea is that **each recursive call** is responsible for solving the maze from **one particular position**.



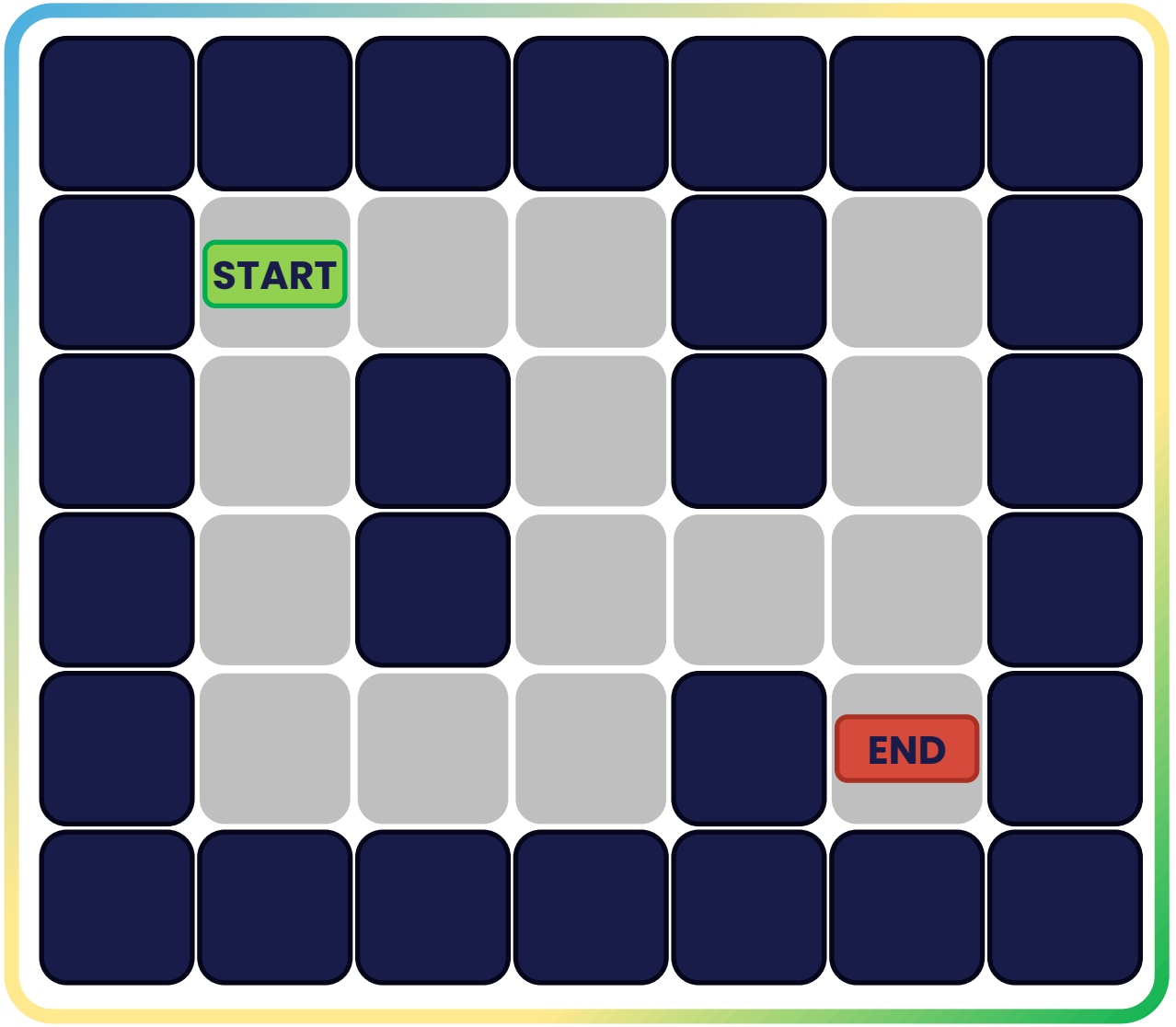
The Base Cases

There are two conditions that stop further recursion.

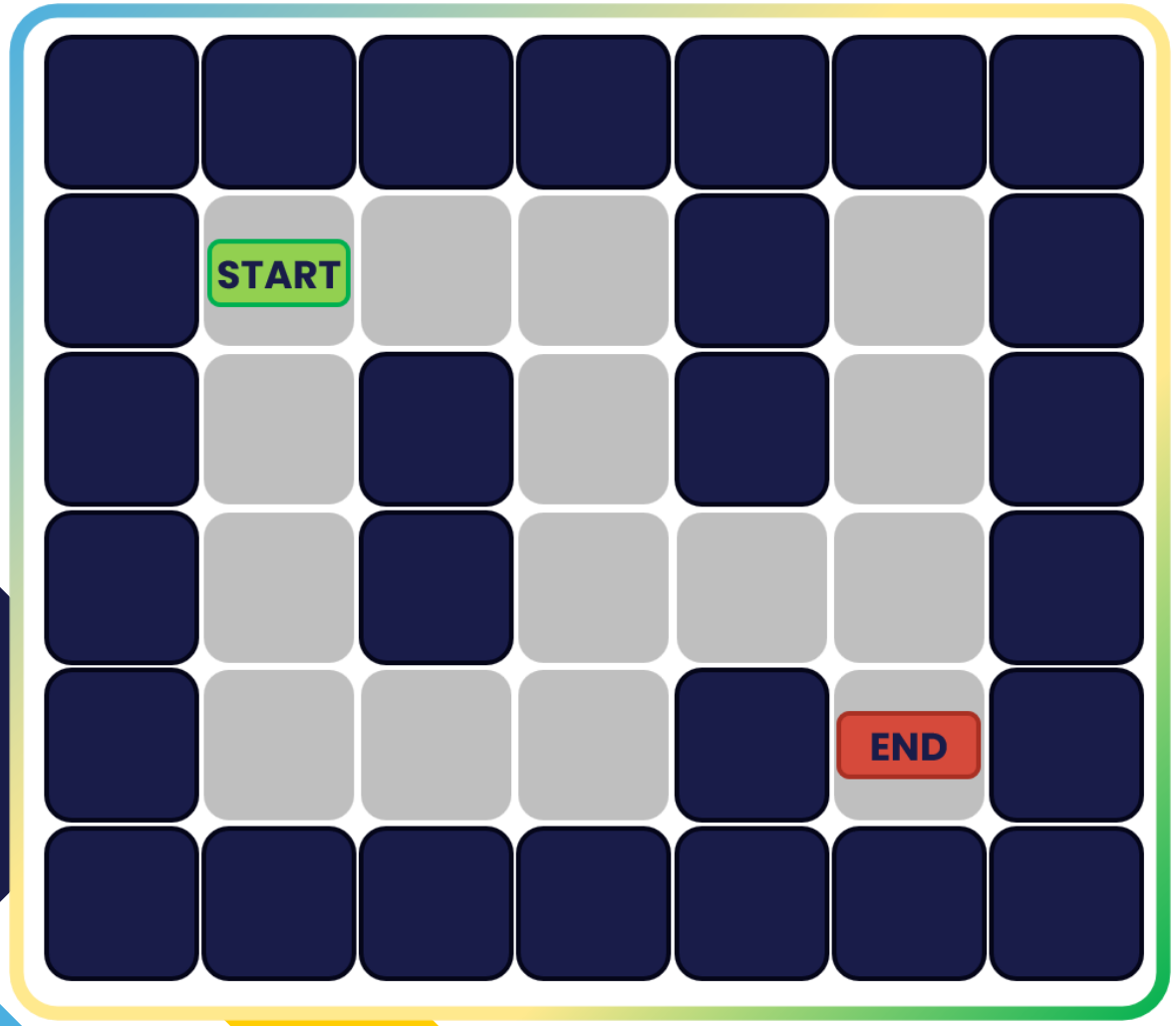
Successful base case: The current position is the exit. Return `True`.

Unsuccessful base case: The current position is invalid or has already been visited. Return `False`.

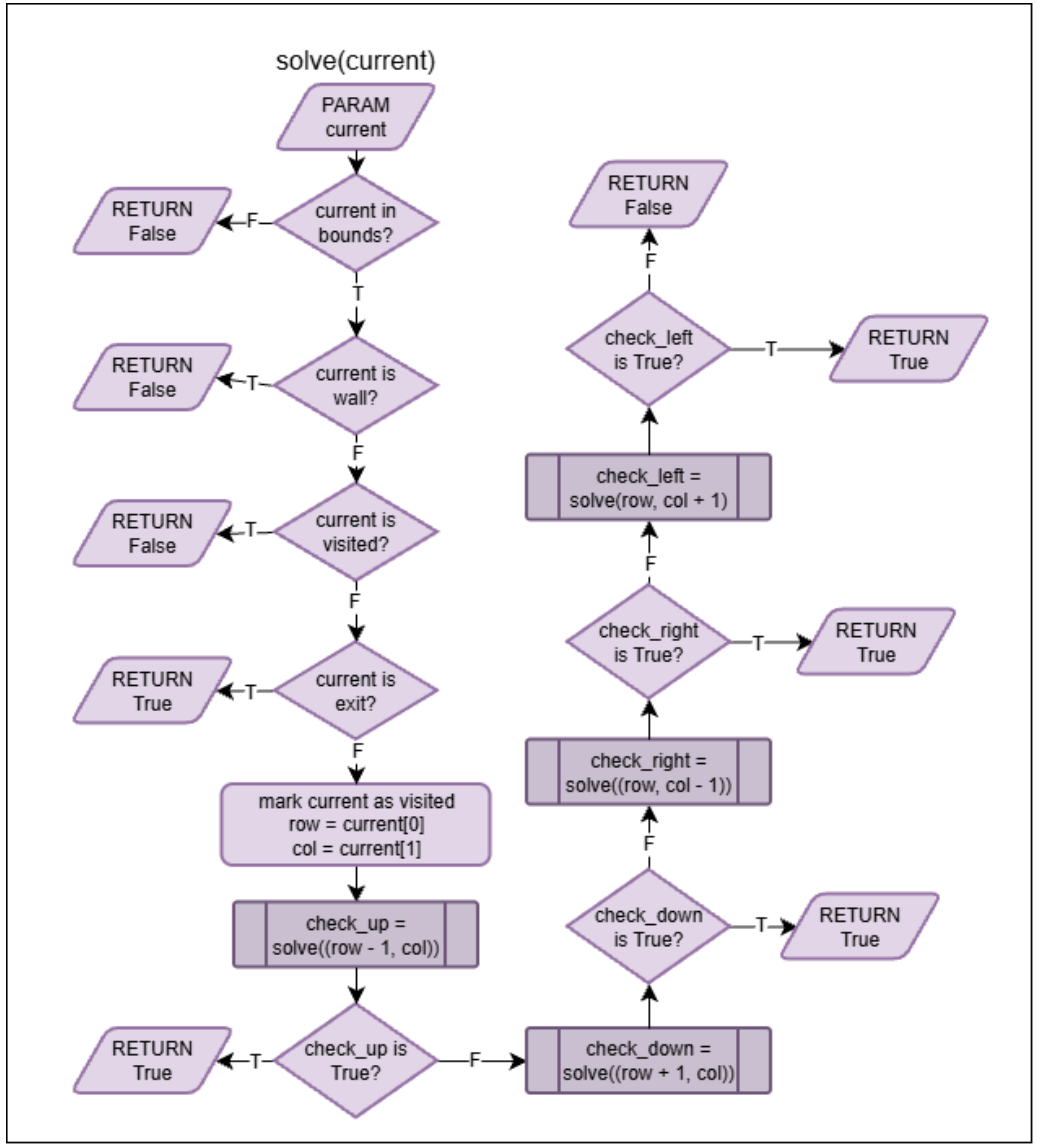
If neither condition is met, the function attempts to solve the maze from another position.



Depth First Search Algorithm



View Simulation

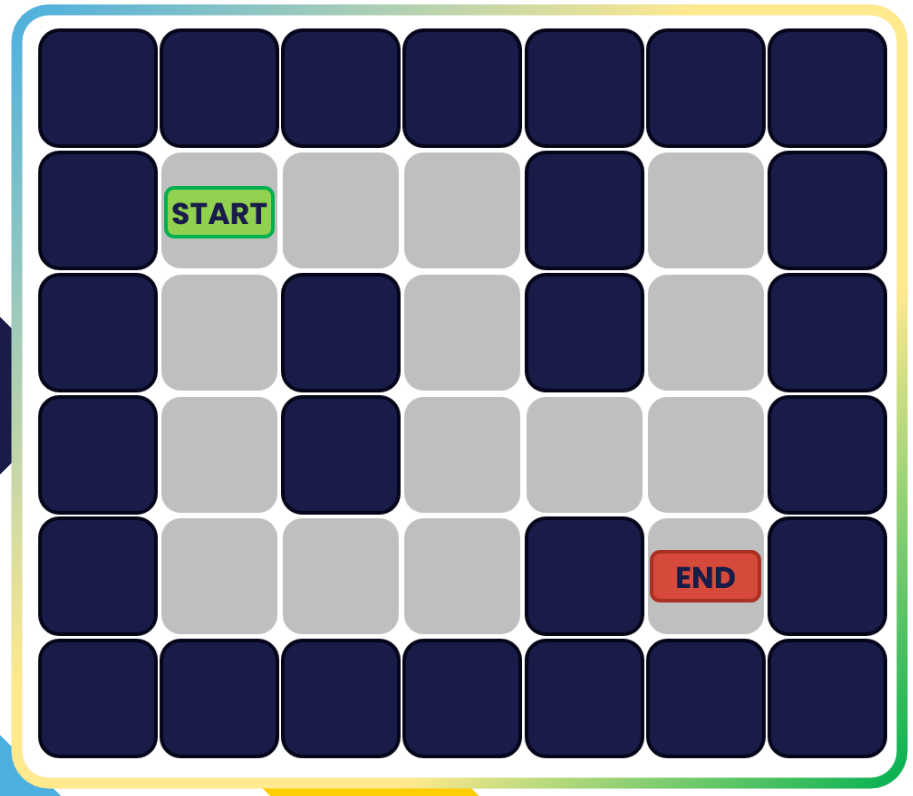


Representing Maze Data

Before getting too far ahead of ourselves, we need to identify how to store maze data.

Here we use a 2D list where `1` represents walls and `0` represents empty squares.

We also need to define our `start` and `end` point.




```
maze = [
    [1, 1, 1, 1, 1, 1, 1],
    [1, 0, 0, 0, 1, 0, 1],
    [1, 0, 1, 0, 1, 0, 1],
    [1, 0, 1, 0, 0, 0, 1],
    [1, 0, 0, 0, 1, 0, 1],
    [1, 1, 1, 1, 1, 1, 1]
]

start = (1, 1)
goal = (4, 5)
```

Representing Maze Data

A location in the maze is identified using two values: its **row** and **column**.

For example, `maze[1][1]` refers to the starting position.

To move **left** or **right**, we change our **column value**.

To move **up** or **down**, we change our **row value**.

```
maze = [
    [1, 1, 1, 1, 1, 1, 1],
    [1, 0, 0, 0, 1, 0, 1],
    [1, 0, 1, 0, 1, 0, 1],
    [1, 0, 1, 0, 0, 0, 1],
    [1, 0, 0, 0, 1, 0, 1],
    [1, 1, 1, 1, 1, 1, 1]
]

start = (1, 1)
goal = (4, 5)
```

Representing Visited Spaces

As our algorithm explores the maze, we need to keep track of which spaces have already been visited.

If we don't track visited spaces, our algorithm could repeatedly travel between the same positions and never finish.

We can represent visited spaces by changing their value in our 2D list.

For example:

- `0` **= Open space**
- `1` **= Wall**
- `2` **= Visited space**

A visited space should not be explored again.

```
maze = [
    [1, 1, 1, 1, 1, 1, 1],
    [1, 0, 0, 0, 1, 0, 1],
    [1, 0, 1, 0, 1, 0, 1],
    [1, 0, 1, 0, 0, 0, 1],
    [1, 0, 0, 0, 1, 0, 1],
    [1, 1, 1, 1, 1, 1, 1]
]

start = (1, 1)
goal = (4, 5)
```

Moving Through the Maze

Our algorithm needs to explore the spaces surrounding its current position.

For any given position, there are four possible directions we can move: up, down, left, and right.

Each movement changes either the row or column of our current position.

How would we calculate the coordinates of the spaces below, left, and right of our current position?

```
# Current position
row = 3
col = 4

# Position above
new_row = row - 1
new_col = col
```

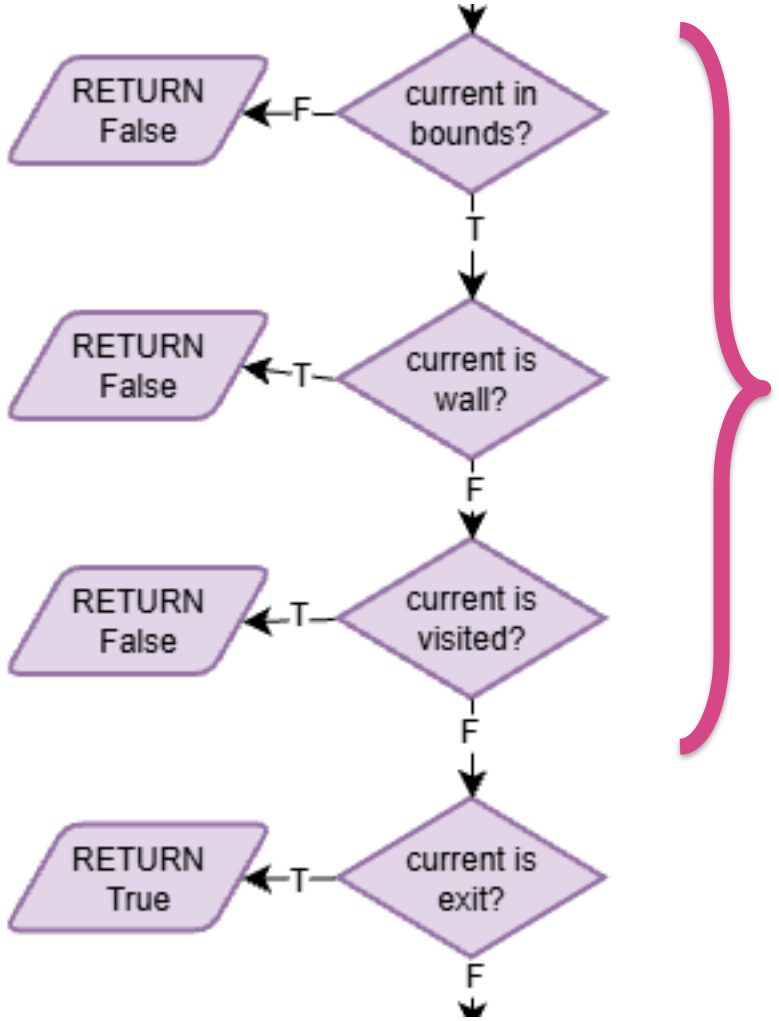
Identifying Invalid Positions

Before exploring a position, our algorithm must determine whether it is valid.

A position is invalid if it is outside the maze, contains a wall, or has already been visited.

We can check these conditions using selection statements.

If the position is invalid, our function should immediately return False.



Identifying the Exit

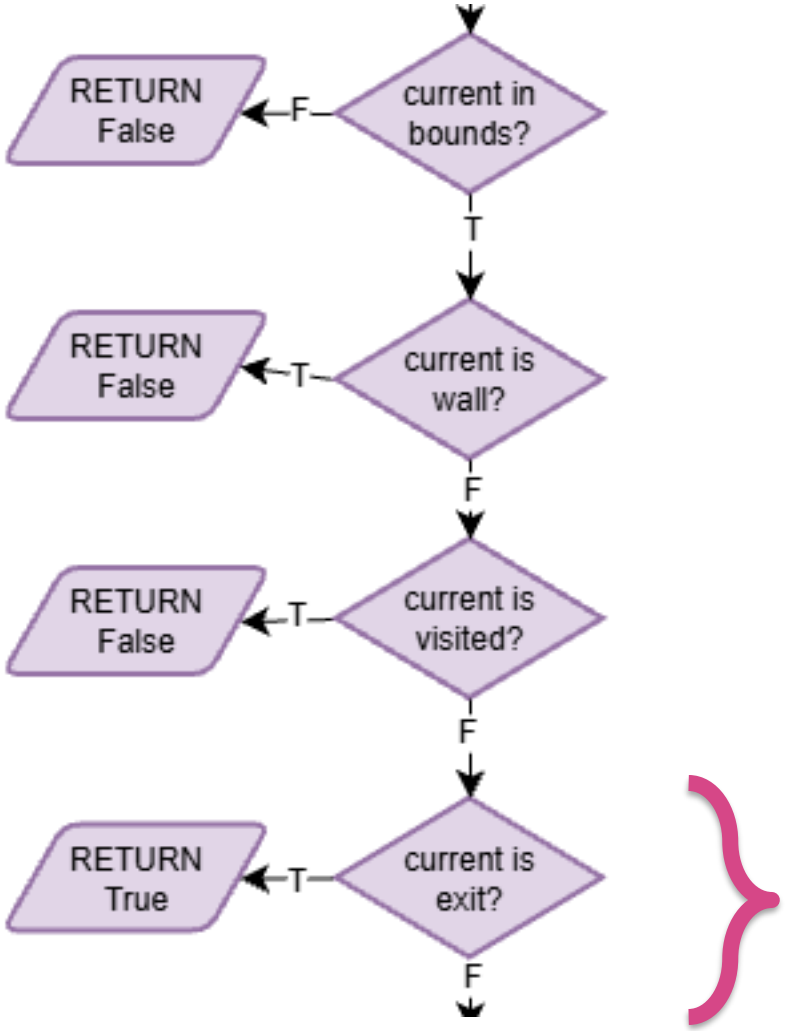
Our algorithm needs to recognize when it has successfully reached the exit.

We already know the coordinates of our destination because they are stored in the goal variable.

```
goal = (4, 5)
```

We can compare our current position against these coordinates.

If our current position matches the goal, our function should return `True`.



Understanding Backtracking

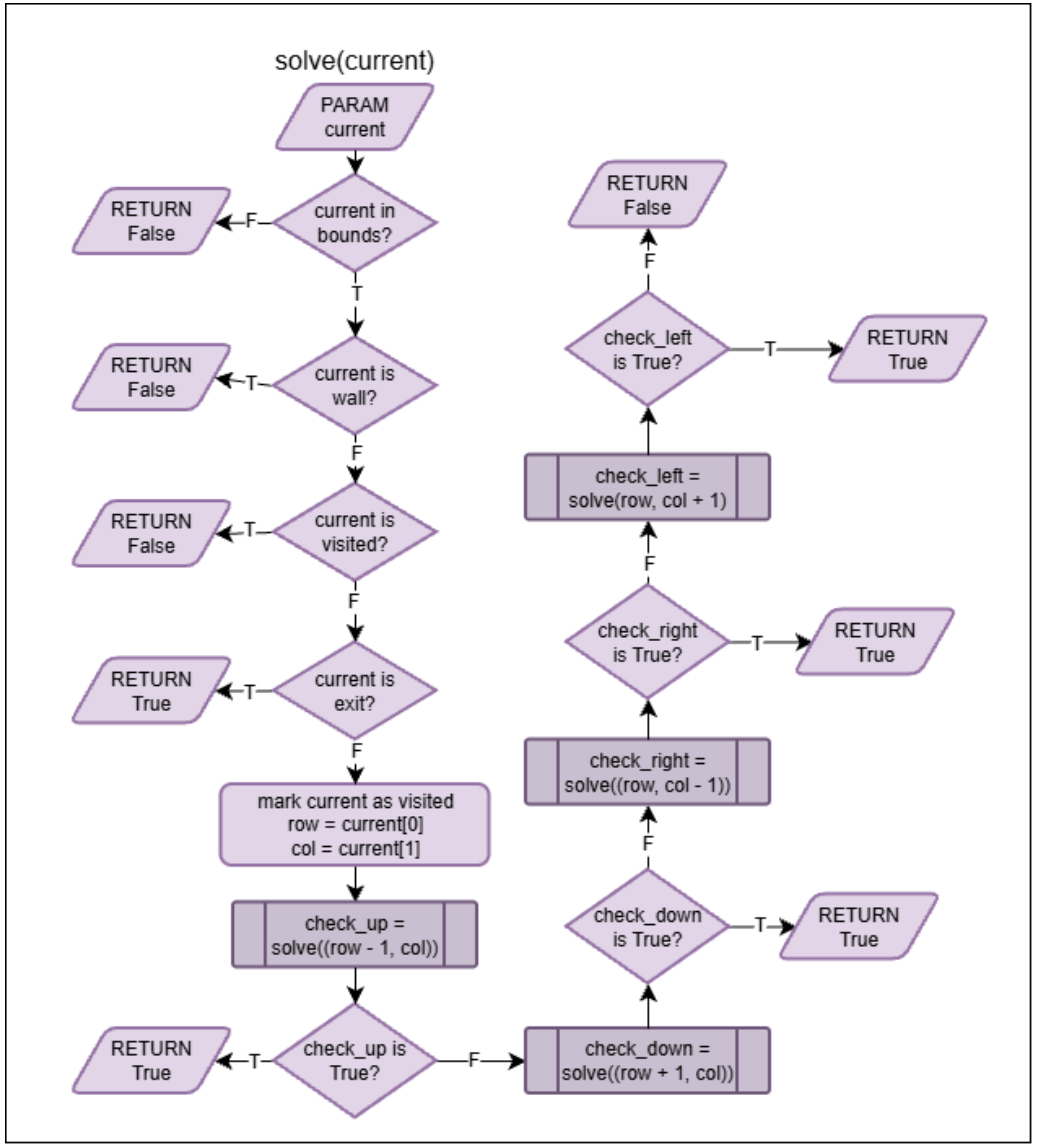
When a recursive call reaches a dead end, it returns `False`.

Python removes that function call from the **call stack** and resumes the previous call.

The previous call can then try another direction.

This process is called **backtracking**.

It allows the algorithm to explore alternative routes when its current route fails.



Combining Our Recursive Calls

Our algorithm must recursively explore the four neighbouring positions.

- If any recursive call returns `True`, the current function returns `True`.
- If a recursive call returns `False`, the function tries another direction.
- If all four directions return `False`, the current function returns `False`.

This allows successful results to travel back through the call stack to the original function call.

